

Project Proposal: Text-FixIt for Repair, Search, and Backtracking

1 Motivation

The long-term goal is to build a general reasoning agent for physical problem solving. Such an agent should be able to propose a candidate repair or action, observe the outcome, recognize that a branch has failed, backtrack to the relevant decision point, and try a more targeted alternative. This project studies that capability in a simplified, text-only version of the FixIt setting.

The original FixIt task studies malfunctioning 3D objects and asks which part transformation would restore functionality. We restrict all examples in this proposal to the original FixIt object categories: Refrigerator, Bucket, USB, Kettle, Cart, KitchenPot, and Box. In this project, the agent proposes candidate repairs and observes the resulting physical outcome. A fix is represented as

$$\text{fix} = (\text{part}, \text{transformation}).$$

Rather than beginning with rendered images, videos, or real robot control, this project abstracts the problem into a cheap text-based environment. The agent receives a symbolic description of the object, a natural-language goal, and simulator-derived final states after each attempted fix. It must choose the next fix to simulate or commit to a final repair.

The central research question is:

Can a language model trained with supervised traces and reinforcement learning learn to search over part–transformation repairs, use outcome feedback, and backtrack and choose alternative repairs efficiently?

2 Project Summary

We will build **Text-FixIt**, a Gym-style text environment inspired by FixIt. Each episode presents a malfunctioning object with named parts and a set of available repair operations. The model can call a simulated repair tool. The simulator applies the proposed fix, activates the object, and returns a compact text/JSON observation containing the final positions, orientations, and task score.

The agent’s job is to interact with this environment:

1. Read the object description and goal.
2. Propose a structured repair action.
3. Observe the final state and score after the fix is applied and the object is activated.
4. Decide whether to backtrack to a different part or transformation, simulate another fix, or choose a final fix.

This project intentionally starts with a pure text version. No VLM, images, videos, or robot data are required for the first milestone.

3 Environment Design

3.1 Episode State

Each episode contains:

- an object category, drawn from the original FixIt categories: Refrigerator, Bucket, USB, Kettle, Cart, KitchenPot, and Box;
- a natural-language functional goal;
- a list of parts;
- an initial symbolic state for each part;
- a hidden malfunction and hidden correct repair;
- a history of previously attempted fixes and simulator outcomes.

Example initial state:

```
Object: Cart
Goal: roll forward while carrying the load.

Parts:
- platform: center=[0.00, 0.00, 0.20], size=[0.60, 0.35, 0.05], rotation_z=0
- front_left_wheel: center=[0.25, -0.15, 0.08], rotation_z=18
- front_right_wheel: center=[0.25, 0.15, 0.08], rotation_z=-18
- rear_left_wheel: center=[-0.25, -0.15, 0.08], rotation_z=0
- rear_right_wheel: center=[-0.25, 0.15, 0.08], rotation_z=0
- load: center=[0.00, 0.00, 0.30]
- target: center=[1.00, 0.00, 0.20]

Available repair operations: defined later in the Transformation
Representation subsection.

History: none
```

3.2 Transformation Representation

This subsection gives the formal repair representation. Each repair is represented as

$$\text{fix} = (\text{part}, \text{transformation}).$$

The `part` field names the object component to modify, and the `transformation` field is a numeric geometric edit applied to that part. In the initial implementation, transformations are restricted to translations and rotations:

- **Translation:** `translate(delta_xyz=[dx, dy, dz])`, where the vector specifies the 3D displacement applied to the selected part.
- **Rotation:** `rotate(axis=[ax, ay, az], angle_deg=theta)`, where the axis specifies the rotation axis and `angle_deg` specifies the rotation angle in degrees.

For example:

```
(part = front_left_wheel,  
transformation = rotate(axis=[0,0,1], angle_deg=-18))
```

or

```
(part = handle,  
transformation = translate(delta_xyz=[0.00, 0.05, 0.00]))
```

3.3 Action Space

The model outputs structured JSON actions. The core actions are:

```
{"action": "choose_fix",  
 "part": "front_left_wheel",  
 "transformation": {"type": "rotate", "axis": [0,0,1], "angle_deg": -18}}  
  
{"action": "backtrack", "to": "part_selection"}  
  
{"action": "backtrack", "to": "transformation_selection"}
```

The fix has exactly two conceptual fields: the part and the transformation.

For the initial version, the backtrack action may be used only as an explicit reasoning/action label during SFT. In the RL environment, it can either be included as a low-cost cognitive action or omitted, forcing the model to express backtracking by choosing a different fix.

3.4 Simulator Output

To keep feedback automatic, the simulator does not generate semantic language such as “the cart veered sideways after the repair.” Instead, it returns the attempted fix, final object states, and a scalar score.

Example simulator result:

```
{  
  "attempted_fix": {  
    "part": "platform",  
    "transformation": {"type": "translate", "delta_xyz": [0.00, 0.08, 0.00]}  
  },  
  "score": 0.24,  
  "parts_initial": {  
    "platform": {"center": [0.00,0.00,0.20], "rotation": [0,0,0], "size":  
      [0.60,0.35,0.05]},  
    "front_left_wheel": {"center": [0.25,-0.15,0.08], "rotation": [0,0,18]},  
    "front_right_wheel": {"center": [0.25,0.15,0.08], "rotation": [0,0,-18]},  
    "load": {"center": [0.00,0.00,0.30]},  
    "target": {"center": [1.00,0.00,0.20]}  
  },  
  "parts_final": {  
    "platform": {"center": [0.24,0.18,0.20], "rotation": [0,0,12]},  
    "front_left_wheel": {"center": [0.49,0.02,0.08], "rotation": [0,0,18]},  
    "front_right_wheel": {"center": [0.50,0.31,0.08], "rotation": [0,0,-18]},  
    "load": {"center": [0.22,0.20,0.30]},  
    "target": {"center": [1.00,0.00,0.20]}  
  }  
}
```

```
}

```

This is deliberately low-level and automatic. The agent must infer what to do next from the state change and score.

4 Task Families

The first version should include only the seven object categories used in the original FixIt benchmark: Refrigerator, Bucket, USB, Kettle, Cart, KitchenPot, and Box. Each category is converted into one or more text tasks with named parts, allowable numeric transformations, and hidden malfunction causes.

Original FixIt Category	Functionality Goal	Example Numeric Fixes
Refrigerator	Door opens/closes and storage region remains accessible	rotate door; translate handle; trans
Bucket	Holds contents without spilling and remains usable	translate wall upward; rotate handl
USB	Connector can be inserted and exposed correctly	translate connector outward; rotate
Kettle	Holds liquid and pours toward target	translate spout outward; rotate spo
Cart	Rolls forward while carrying load	rotate wheel about z-axis; translate
KitchenPot	Holds contents and supports manipulation by handles/lid	translate wall upward; rotate lid; tr
Box	Opens/closes or stores contents correctly	rotate lid; translate hinge; translate

Table 1: Text-FixIt-RL uses only object categories from the original FixIt benchmark. Transformations are numeric translations or rotations, not symbolic repair labels.

5 Training Plan

5.1 Stage 1: Off-the-shelf LLMs or VLMs, no training

We first try off-the-shelf LLMs or VLMs (with no training) on both the image-based and text-based version of this task, to verify that it is not already solved.

5.2 Stage 2: Supervised Fine-Tuning

Generate oracle or search-derived traces. Each trace should include failed attempts, state feedback, backtracking, and final success.

Example trace:

```
State: Cart, goal=roll forward while carrying the load.

Action 1:
{"action": "simulate_fix",
 "part": "platform",
 "transformation": {"type": "translate", "delta_xyz": [0.00, 0.08, 0.00]}}

Observation 1:
score=0.24, cart_forward_displacement=0.24, cart_lateral_drift=0.18,
  cart_distance_to_target_final=0.78

Action 2:
{"action": "backtrack", "to": "part_selection"}
```

```

Action 3:
{"action": "simulate_fix",
 "part": "front_left_wheel",
 "transformation": {"type": "rotate", "axis": [0,0,1], "angle_deg": -18}}

Observation 3:
score=0.88, cart_forward_displacement=0.83, cart_lateral_drift=0.02,
  cart_distance_to_target_final=0.17

Action 4:
{"action": "choose_fix",
 "part": "front_left_wheel",
 "transformation": {"type": "rotate", "axis": [0,0,1], "angle_deg": -18}}

```

The SFT objective is to train the model to output valid structured actions conditioned on the current search state.

5.3 Stage 3: Reinforcement Learning

After SFT, train the model in the environment with a reward such as:

$$R = R_{\text{final}} - \lambda_{\text{sim}} N_{\text{sim}} - \lambda_{\text{repeat}} N_{\text{repeat}} - \lambda_{\text{invalid}} N_{\text{invalid}}.$$

A simple reward instantiation:

- +1.0 for choosing a successful or high-score final fix;
- −0.05 per simulator call;
- −0.20 for simulating the same fix twice;
- −0.50 for choosing a low-score fix;
- −0.05 for malformed JSON or invalid action.

The RL action space should remain structured. The model chooses a part and a numeric transformation object.

6 Model Choice

For the text-only version, we should not start with a VLM. The input is text/JSON, so a text LLM is simpler and more appropriate.

Recommended model ladder:

- **Debugging:** Qwen2.5-1.5B-Instruct or a similar small instruction model.
- **Main experiments:** Qwen2.5-7B-Instruct or Qwen3-8B-Instruct with LoRA SFT and RL.
- **Future visual extension:** Qwen2.5-VL-7B-Instruct or Qwen3-VL once rendered object views or videos are added.

The output format should be constrained to JSON actions to make parsing and environment interaction reliable.

7 Evaluation

Report the following metrics:

- final fix success rate;
- average final functionality score;
- number of simulator calls per solved episode;
- recovery rate after an initially wrong fix;
- repeated-failure rate;
- invalid-action rate;
- generalization to held-out object categories, malfunction causes, and part–transformation combinations.

Key ablations:

1. final-answer-only SFT: train only to output the correct fix;
2. trace SFT without failed branches;
3. trace SFT with failed branches but no explicit backtrack labels;
4. trace SFT with explicit backtrack labels;
5. RL after SFT;
6. RL without SFT warm start.

The central hypothesis is that SFT on failed-branch search traces, followed by RL, will improve recovery after wrong initial attempts and reduce the number of simulator calls required to find a successful repair.

8 Expected Outcome

A successful project would demonstrate that a text LLM can learn a structured repair-search policy in a FixIt-inspired environment. The strongest result would show that models trained on failed-branch traces and improved with RL outperform one-shot predictors and greedy search, especially on recovery after an initially wrong repair.

This would provide a low-cost foundation for later visual and robotics versions:

Text-FixIt → Rendered FixIt with VLM → Robot action-plan repair.

9 Stretch Goals

If time permits:

- add rendered object views and switch to Qwen-VL;
- use short state histories instead of final states only;
- add multi-fix tasks requiring two repairs;

References

- [1] Y. Hong, K. Mo, L. Yi, L. J. Guibas, A. Torralba, J. B. Tenenbaum, and C. Gan. Fixing Malfunctional Objects with Learned Physical Simulation and Functional Prediction. CVPR 2022; arXiv:2205.02834.
- [2] Qwen Team. Qwen2.5 Technical Reports and Model Cards. 2024–2025.